INSTITUTO FEDERAL CATARINENSE - IFC
BACHARELADO DE CIÊNCIA DA COMPUTAÇÃO

ANDRÉ VITOR BASTOS DE MACÊDO

ALGORITMOS DE BUSCA DE CAMINHO EM REPRESENTAÇÕES NAVEGÁVEIS TRIDIMENSIONAIS : Uma análise estatística e comparativa

Blumenau
2026

ANDRÉ VITOR BASTOS DE MACÊDO

ALGORITMOS DE BUSCA DE CAMINHO EM REPRESENTAÇÕES NAVEGÁVEIS TRIDIMENSIONAIS : Uma análise estatística e comparativa

Trabalho de Conclusão de Curso apresentado ao curso de Bacharelado em Ciência da Computação do Instituto Federal Catarinense.

Orientador(a): Prof. Paulo César Rodacki Gomes

Blumenau
2026

LISTA DE ILUSTRAÇÕES

Figura 1 Comparação entre representações 2D e 3D 7
Figura 2 Exemplo de grafo simples. 9
Figura 3 Exemplo de grafo direcionado e não direcionado. 11
Figura 4 Exemplo de grafo ponderado e não ponderado. 12
Figura 5 Exemplo de caminho em um grafo. 13
Figura 6 Exemplo de componentes conexos em um grafo. 13
Figura 7 Diversos caminhos possíveis entre dois vértices em um grafo. 14
Figura 8 Pseudocódigo do algoritmo de Dijkstra. 14
Figura 9 Pseudocódigo do algoritmo de Dijkstra ponto a ponto. 15
Figura 10 Pseudocódigo do algoritmo A*. 17
Figura 11 Pseudocódigo do algoritmo Weighted A*. 18
Figura 12 Pseudocódigo da Greedy Best-First Search. 19
Figura 13 Conversão de um mapa de altura em vértices tridimensionais. 20
Figura 14 Exemplo de malha gerada a partir de um mapa de altura 21
Figura 15 Ruído de Perlin com células de diferentes tamanhos. 22
Figura 16 Variação da lacunaridade: ruídos com lacunaridade baixa e alta. 23
Figura 17 Variação da persistência: ruídos com persistência baixa e alta. 24
Figura 18 Variação do número de oitavas: ruídos com 1 e 6 oitavas. 25
Figura 19 Exemplo de reconstrução de superfície com Marching Cubes. 26
Figura 20 Exemplo de grade 2.5D. 26
Figura 21 Exemplo de grade voxelizada. 27
Figura 22 Exemplo de grafo de vértices. 27
Figura 23 Exemplo de grafo dual. 27
Figura 24 Etapas do pipeline experimental. 29
Figura 25 Diagrama de classes do padrão *Composite* nas malhas do IFCG. 30
Figura 26 Pool de threads e filas de prioridade do *TaskMaster*. 31
Figura 27 Cena renderizada pelo visualizador do TCC. 31
Figura 28 Representação do `lwGraph`: vetor de vértices e vetores de adjacência. . 32
Figura 29 Diferentes representações navegáveis do mesmo terreno. 34

LISTA DE TABELAS

Tabela 1 Variáveis independentes e dependentes do experimento. 40

LISTA DE ABREVIATURAS E SIGLAS

SIG Sigla

SUMÁRIO

1 INTRODUÇÃO 7
1.1 Contribuições 8
1.2 Objetivos 8
2 FUNDAMENTAÇÃO TEÓRICA 9
2.1 Grafos 9
2.1.1 Definição 9
2.1.2 Direcionamento 11
2.1.3 Pesos 11
2.1.4 Caminho 12
2.1.5 Componente conexo 13
2.2 Busca 14
2.2.1 Dijkstra 14
2.2.2 Heurísticas 16
2.2.3 A* 17
2.2.4 Weighted A* 18
2.2.5 Greedy Best-First Search 19
2.3 Geração Procedural de Terrenos 20
2.3.1 Mapas de altura 20
2.3.2 Ruído de Perlin 21
2.4 Representações Geométricas de Terrenos 25
2.4.1 Voxels 25
2.4.2 Marching Cubes 25
2.5 Representações Navegáveis de Terrenos 26
2.5.1 Grade 2.5D 26
2.5.2 Grade Voxelizada 27
2.5.3 Grafo de Vértices 27
2.5.4 Grafo Dual 27
2.6 Avaliação de desempenho e estatísticas 28
2.6.1 ANOVA 28
2.6.2 Teste post hoc de Tukey 28
2.7 Trabalhos relacionados 29
2.7.1 Johansson 29
2.7.2 Kapi 29
2.7.3 Matriz de diferença 29
2.8 Pipeline 29
2.9 IFCG e visualizador concorrente 29
2.9.1 Arquitetura e *Singleton* 29
2.9.2 Janela e contexto 30
2.9.3 Renderização e laço principal 30
2.9.4 Malhas e composição 30

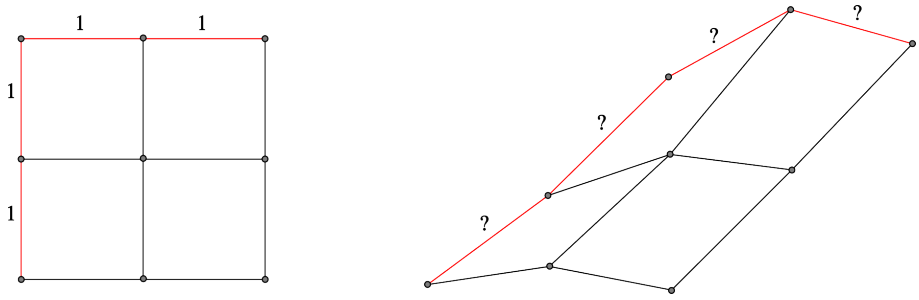
2.9.5 Shaders e buffers 30
2.9.6 Câmera e entrada 31
2.9.7 Concorrência e *TaskMaster* 31
2.10 lwGraph<T> 32
2.11 search.hpp 32
2.11.1 Algoritmos 33
2.11.2 Aquisição de dados 33
2.12 Gerador de terrenos 34
2.13 GraphModel e GraphFactory 34
2.14 Instrumentação 35
2.14.1 Procedimento dos testes 36
2.14.2 Statistics 36
2.14.3 coleta_completa.sh 37
2.14.4 Interface de linha de comando (CLI) 38
3 METODOLOGIA 39
3.1 Tipo de pesquisa 39
3.2 Ambiente 39
3.3 Variáveis 39
3.4 Níveis 40
3.5 Procedimento e métricas 40
3.6 Protocolo estatístico 41
3.7 Ameaças 41
4 RESULTADOS E DISCUSSÃO 43
5 CONCLUSÃO 44
REFERÊNCIAS 45

1 INTRODUÇÃO

A busca de caminhos (ou *pathfinding*) é um dos campos mais tradicionais da Inteligência Artificial aplicada, evoluindo a partir dos algoritmos de busca clássicos em grafos (ver Seção 2.1). Como definem Russell; Norvig (2013), os métodos clássicos de busca resolvem problemas de estados, ações e custos, visando encontrar caminhos entre um ponto de partida e um objetivo por meio da exploração do espaço de busca. Ao longo das últimas décadas, essa fundamentação deixou de ser puramente abstrata e passou a ser aplicada diretamente na computação interativa. Conforme apontam Pardede et al. (2022), os algoritmos de busca tornaram-se componentes cruciais para o desenvolvimento de jogos modernos e simulações em tempo real.

No entanto, a transição do espaço de busca de 2D para 3D introduziu um aumento significativo na complexidade, exigindo adaptações às técnicas de busca e de representação do terreno. A escolha da representação do espaço transitável e do algoritmo de busca a ser utilizado tornou-se uma decisão crítica, impactando diretamente o desempenho e a qualidade dos caminhos encontrados.

Figura 1 – Comparação entre representações 2D e 3D



Fonte: Elaborado pelo autor.

Em estudos recentes, é comumente evitada a comparação de múltiplas representações navegáveis de terrenos 3D, com foco em apenas uma abordagem na maioria das vezes (Johansson; Others, 2024; Kapi; Sunar; Algfoor, 2022). Além disso, a análise estatística do desempenho dos algoritmos de busca em diferentes representações é frequentemente negligenciada, limitando a compreensão das interações entre a estrutura do terreno e o comportamento dos algoritmos.

Esse trabalho propõe uma análise comparativa e estatística do desempenho de algoritmos clássicos de busca de caminhos em diferentes representações navegáveis de terrenos 3D. A pesquisa busca responder à seguinte questão central: Qual é a influência da **representação navegável** de um terreno 3D e das **características do terreno** no desempenho e na qualidade da busca de algoritmos clássicos de busca?

1.1 Contribuições

Para responder à questão central, é necessário realizar uma análise detalhada das representações navegáveis de terrenos 3D (ver Seção 2.5), dos algoritmos de busca aplicados (ver Seção 2.2) e das métricas de desempenho utilizadas (ver Seção 2.6). Com isso, as principais contribuições deste trabalho incluem:

1. **Comparação entre representações navegáveis.**
   As mesmas instâncias de terreno são convertidas em quatro grafos diferentes e avaliadas com os mesmos algoritmos:
   - a partir do mapa de altura: grade de altura (2.5D) e grade voxelizada;
   - a partir da malha poligonal: grafo de vértices e grafo dual.
2. **Metodologia experimental reprodutível.**
   Os terrenos são gerados proceduralmente (ver Seção 2.3) a partir de parâmetros controlados (escala, lacunaridade, persistência e comprimento de onda) e semente registrada, e os resultados passam por análise estatística (análise de variância, ou ANOVA, e testes post hoc).
3. **Implementação open-source.**
   Inclui uma biblioteca de grafos leve e o motor gráfico IFCG, com visualização das malhas e dos caminhos.

1.2 Objetivos

O objetivo geral do estudo é analisar comparativa e estatisticamente o desempenho de algoritmos clássicos de *pathfinding* em diferentes representações navegáveis de terrenos 3D. Mais especificamente, os objetivos incluem:

1. Implementar uma biblioteca de grafos e os algoritmos Dijkstra, A*, Weighted A* e Greedy Best-First Search.
2. Desenvolver um gerador procedural de terrenos baseado em ruído de Perlin, parametrizado por escala, lacunaridade, persistência e comprimento de onda.
3. Construir quatro representações navegáveis a partir do terreno: grade de altura, grade voxelizada, grafo de vértices e grafo dual da malha.
4. Desenvolver um ambiente 3D com execução concorrente para visualização das malhas e dos caminhos encontrados.
5. Automatizar a coleta de métricas e analisar estatisticamente os resultados obtidos.

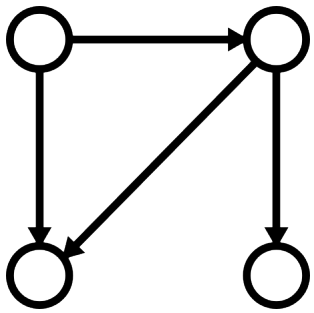
2 FUNDAMENTAÇÃO TEÓRICA

A comparação proposta neste trabalho depende de três conjuntos de conceitos: a teoria de grafos, que modela o espaço de busca; os algoritmos que percorrem esse modelo; e os métodos de representação de terrenos que determinam como o grafo é construído a partir da geometria. Este capítulo apresenta cada um deles, nessa ordem, e termina com os trabalhos que compararam representações semelhantes.

2.1 Grafos

Grafos são estruturas matemáticas que representam relações entre objetos. Eles consistem em um conjunto de vértices (ou nós) e um conjunto de arestas (ou ligações) que conectam os vértices. Um grafo pode ser direcionado ou não direcionado, ponderado ou não ponderado, dependendo das características do problema a ser modelado.

Figura 2 – Exemplo de grafo simples.



Fonte: Elaborado pelo autor.

Quando trabalhamos com estruturas de dados complexas, como terrenos 3D, os grafos fornecem uma representação abstrata que facilita a aplicação de algoritmos de busca. Em aplicações como a navegação em terrenos 3D, os vértices podem representar posições do ambiente, e as arestas, a transitabilidade entre elas.

2.1.1 Definição

Segundo Gomes (2022), um grafo G é definido como um par ordenado (Equação 1), onde: G é o grafo; V é o conjunto de vértices; e A é o conjunto de arestas. Cada aresta representa a conexão entre dois vértices, podendo ter uma direção associada e um peso.

$$G = (V, A) \tag{1}$$

Cada conjunto de vértices e arestas pode ser representado de diferentes maneiras, dependendo da aplicação e das operações que serão realizadas sobre o grafo. As representações mais comuns incluem listas de adjacência (Seção 2.1.1.3) e matrizes de adjacência (Seção 2.1.1.4) (Gomes, 2022).

Dado que um grafo é uma estrutura de dados baseada em conjuntos, naturalmente existem subgrafos. Um subgrafo é formado por um subconjunto de vértices e por um subconjunto de arestas cujas extremidades pertencem a esse subconjunto de vértices, podendo ser utilizado para analisar partes específicas da estrutura ou para simplificar a resolução de problemas complexos (Gomes, 2022).

2.1.1.1 Vértices

Vértices são as unidades básicas de um grafo, também chamadas de nós. Cada vértice funciona como um elemento unitário que pode carregar qualquer tipo de valor ou informação. Dependendo da aplicação, um vértice pode representar um espaço na memória, um local, uma ideia ou um estado, como a configuração de um sistema em um dado momento. A definição de grafo não restringe o significado dos vértices, que depende apenas do problema modelado.

2.1.1.2 Arestas

Arestas são as conexões que ligam pares de vértices de um grafo. Cada aresta representa uma relação entre dois vértices, e seu significado depende da aplicação. Uma aresta pode representar uma passagem física entre dois locais, uma passagem lógica entre dois estados ou uma mudança de estado, como a transição de um sistema de uma configuração para outra.

2.1.1.3 Lista de adjacência

Lista de adjacência é uma forma de representar um grafo em que cada vértice armazena a lista dos vértices diretamente conectados a ele por uma aresta. Para um grafo com n vértices e m arestas, essa estrutura ocupa espaço proporcional a $n + m$, o que a torna eficiente para grafos com poucas arestas (Cormen et al., 2009).

2.1.1.4 Matriz de adjacência

Matriz de adjacência é uma forma de representar um grafo por meio de uma tabela quadrada de dimensão $n \times n$, em que cada linha e cada coluna correspondem a um vértice. Cada posição da matriz indica se existe uma aresta entre o par de vértices correspondente, e pode também armazenar o peso dessa aresta. Em um grafo não direcionado, a matriz é simétrica, pois a conexão vale nos dois sentidos.

Essa estrutura ocupa espaço proporcional a n^2, o que a torna pouco eficiente para grafos com poucas arestas. Em compensação, ao contrário da lista de adjacência (Seção 2.1.1.3), ela permite verificar em tempo constante se dois vértices estão conectados (Gomes, 2022).

2.1.2 Direcionamento

Como definido em Seção 2.1.1.2, uma aresta conecta dois vértices. Essa conexão pode ter um sentido, caso em que a aresta é direcionada, ou representar apenas uma ligação sem direção. Na criação de um grafo, é possível defini-lo como direcionado (Figura 3.a) ou não direcionado (Figura 3.b). Ou seja, a conexão entre dois vértices pode ser unidirecional ou bidirecional.

Figura 3 – Exemplo de grafo direcionado e não direcionado.



(a) Grafo direcionado (b) Grafo não direcionado

Fonte: Elaborado pelo autor.

A escolha entre grafos direcionados e não direcionados depende do problema a ser modelado. Em muitos casos, como em mapas de ruas ou redes de transporte, as conexões são bidirecionais, enquanto em outros, como fluxos de dados ou hierarquias, as conexões podem ser unidirecionais.

2.1.3 Pesos

Grafos também podem ser ponderados (Figura 4.a), atribuindo valores numéricos às arestas para representar custos ou distâncias entre os vértices.

Figura 4 – Exemplo de grafo ponderado e não ponderado.



(a) Grafo ponderado (b) Grafo não ponderado

Fonte: Elaborado pelo autor.

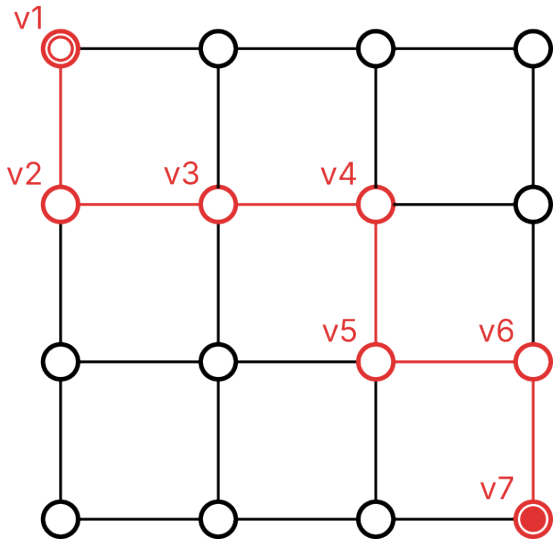
Ponderar a travessia entre vértices permite representar de forma mais precisa as características do ambiente, como a dificuldade de atravessar determinados terrenos ou a distância entre pontos. Como visto na Figura 4, a atribuição de pesos às arestas pode influenciar diretamente a escolha do caminho mais eficiente ou mais curto.

O custo de travessia de uma aresta pode representar diferentes aspectos, como distância física, tempo de viagem ou dificuldade de navegação. A escolha de atribuir pesos às arestas depende do contexto do problema e dos objetivos da estrutura. Alguns algoritmos de busca, como o de Dijkstra, exigem que os pesos sejam não-negativos (Cormen et al., 2009).

2.1.4 Caminho

Um caminho em um grafo é uma sequência de vértices conectados por arestas, representando uma rota entre dois pontos (Gomes, 2022), ilustrado na Figura 5. A definição formal de um caminho é uma sequência de vértices $(v_1, v_2, ..., v_n)$ tal que cada par consecutivo $\{v_i, v_{i+1}\}$ está conectado por uma aresta no grafo.

Figura 5 – Exemplo de caminho em um grafo.



Fonte: Elaborado pelo autor.

Considerando v_1 o vértice inicial e v_7 o vértice final, a sequência de vértices $(v_1, v_2, v_3, v_4, v_5, v_6, v_7)$ representada na Figura 5 é o caminho $v_1 \rightarrow v_7$. Esse caminho é uma das várias rotas possíveis entre os vértices v_1 e v_7, e a escolha do caminho mais eficiente depende dos pesos das arestas e das características do grafo.

Para todo vértice v em um grafo, é possível determinar se existe um caminho para outro vértice u no mesmo grafo. Entretanto, a existência de um caminho não impede a existência de outros caminhos entre v e u, podendo haver múltiplos caminhos com diferentes pesos e características. Quando comparados, os caminhos podem apresentar variações significativas em termos de custo, distância ou dificuldade de travessia. Se existe ao menos um caminho entre v e u, e o grafo não contém ciclos de peso negativo, existe um caminho mínimo ou ótimo entre eles, que é aquele com o menor custo ou peso total.

2.1.5 Componente conexo

Dado um grafo não direcionado, um componente conexo é um subgrafo conexo e maximal, isto é, não pode ser estendido com outros vértices ou arestas do grafo original, e nele existe caminho entre quaisquer pares de vértices (Gomes, 2022).

Figura 6 – Exemplo de componentes conexos em um grafo.

Fonte: Elaborado pelo autor.

A identificação de componentes conexos é fundamental para entender a estrutura do grafo e determinar a viabilidade de encontrar caminhos entre diferentes regiões. Uma vez que um vértice pertence a um componente conexo, é possível garantir que existe um caminho para qualquer outro vértice dentro do mesmo componente.

2.2 Busca

A busca de caminhos em um grafo é o processo de encontrar um caminho entre um vértice inicial e um vértice objetivo, idealmente o de menor custo. Diferentes algoritmos de busca podem ser aplicados, dependendo das características do grafo e dos requisitos do problema.

Figura 7 – Diversos caminhos possíveis entre dois vértices em um grafo.

Fonte: Elaborado pelo autor.

Sabendo da necessidade de encontrar caminhos mínimos, algoritmos de busca são projetados para explorar o grafo de maneiras específicas, visando otimizar a descoberta de caminhos com base em critérios como custo, distância ou heurísticas.

2.2.1 Dijkstra

O algoritmo de Dijkstra, proposto por Dijkstra (1959), é um método clássico da busca de caminhos em grafos ponderados, considerando que todos os pesos das arestas são não-negativos. Ele é capaz de encontrar, para cada vértice alcançável, um caminho mínimo a partir de um vértice inicial (Gomes, 2022). Esse processo gera uma árvore de caminhos mínimos tendo o vértice inicial como raiz.

Na implementação do algoritmo de Dijkstra, é considerado o pseudocódigo apresentado por Gomes (2022). Neste capítulo consideraremos um grafo $G = (V, A)$ ponderado de pesos não-negativos, onde V é o conjunto de vértices e A é o conjunto de arestas.

Figura 8 – Pseudocódigo do algoritmo de Dijkstra.

```
Dijkstra(G, W, s)
1   para cada v ∈ V faça
2       g_v ← inf
3       p_v ← nil
4   g_s ← 0
5   Q ← V
6   S ← ∅
7   enquanto Q ≠ ∅ faça
8       u ← pop(Q)
9       S ← S ∪ {u}
10      para cada vértice v ∈ adj(u) faça
11          se v ∈ Q e [g_v > g_u + W_{u,v}] então
12              g_v ← (g_u + W_{u,v})
13              p_v ← u
```

Fonte: Gomes (2022).

Na Figura 8, o algoritmo de Dijkstra inicializa a distância g_v de todos os vértices a partir do vértice inicial s como infinitos, exceto para o vértice inicial, que tem distância zero para ele mesmo. Em seguida são criadas duas estruturas de dados:

- Q: Uma fila de prioridade mínima (ou *min-PQ*, do inglês *Minimum Priority Queue*) que armazena os vértices ainda não processados ordenados pela menor distância conhecida, de modo que o vértice de menor distância seja sempre o primeiro a sair.
- S: Um conjunto que armazena os vértices cuja distância mínima já foi determinada. Os caminhos são reconstruídos a partir dos predecessores p_v.

O laço principal do algoritmo repete-se enquanto Q não estiver vazia. Em cada iteração, a operação $\mathrm{pop}(Q)$ retorna o vértice u com menor distância estimada g_u e o remove da fila. Em seguida, u é incluído em S. Como os pesos são não-negativos, nenhum caminho que passe por vértices ainda não processados pode reduzir g_u, o que garante que essa distância já é a mínima.

Para cada vértice v adjacente a u (ver Seção 2.1.1.3) que ainda está em Q, verifica-se se passar por u produz um caminho melhor do que o já conhecido. Se $g_v > g_u + W_{u,v}$, atualiza-se g_v para $g_u + W_{u,v}$ e registra-se u como predecessor de v em p_v. Essa operação é chamada de relaxamento da aresta (u, v).

Ao término do laço, para cada vértice alcançável a partir de s, g_v contém a distância mínima entre s e v. Os predecessores p_v permitem reconstruir o caminho mínimo, percorrendo-os do destino até s.

Quando apenas o caminho entre dois vértices interessa, o algoritmo pode ser adaptado para uma busca ponto a ponto, com um vértice objetivo t, como na Figura 9.

Figura 9 – Pseudocódigo do algoritmo de Dijkstra ponto a ponto.

```
Dijkstra(G, W, s, t)
1   para cada v ∈ V faça
2       g_v ← inf
3       p_v ← nil
4   g_s ← 0
5   Q ← {s}
6   S ← ∅
7   enquanto Q ≠ ∅ faça
8       u ← pop(Q)
9       se u = t então retorna o caminho até t por p
10      S ← S ∪ {u}
11      para cada vértice v ∈ adj(u) faça
12          se v ∉ S e [g_v > g_u + W_{u,v}] então
13              g_v ← (g_u + W_{u,v})
14              p_v ← u
15              se v ∉ Q então Q ← Q ∪ {v}
```

Fonte: Elaborado pelo autor, com base em Gomes (2022).

Na Figura 9, as diferenças em relação à Figura 8 estão em negrito. A fila Q começa apenas com s, e cada vértice entra nela na primeira vez em que é alcançado. Por isso, a condição de relaxamento verifica se v ainda não foi finalizado, em vez de verificar se ele está em Q. A busca termina quando t é retirado da fila, já que, nesse momento, g_t é mínimo, e os vértices mais distantes que t não chegam a ser processados. O caminho encontrado até t é o mesmo da versão clássica.

2.2.2 Heurísticas

Uma heurística é uma função $h(n)$ que estima o custo de um caminho entre um vértice n e o objetivo, sem precisar percorrê-lo. Ela orienta a busca para os vértices mais promissores. Uma heurística é admissível quando nunca superestima o custo real até o objetivo (Hart; Nilsson; Raphael, 1968). Já a consistência exige que, para cada aresta, a estimativa do vértice de origem não exceda o custo da aresta somado à estimativa do vértice de destino (Russell; Norvig, 2013).

Em algumas variantes, a heurística recebe um fator de ponderação w, e a função de avaliação passa a ser $f(n) = g(n) + w \cdot h(n)$, em que $g(n)$ é o custo acumulado desde o vértice inicial até n. Com $w = 0$, a heurística não influencia a escolha, e a busca se comporta como o Dijkstra. Valores maiores de w tornam a busca mais orientada pela estimativa, o que pode reduzir o número de vértices explorados à custa da otimalidade do caminho (Pohl, 1970).

A forma de calcular $h(n)$ depende do tipo de movimento permitido no espaço de busca. Em terrenos 3D, a estimativa mais comum é a distância euclidiana, que mede a distância em linha reta entre os vértices:

$$h(n) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2} \tag{2}$$

Em grades com movimentos apenas nos eixos, a distância de Manhattan soma as diferenças em cada coordenada:

$$h(n) = |x_2 - x_1| + |y_2 - y_1| + |z_2 - z_1| \tag{3}$$

A distância de Chebyshev toma a maior diferença entre as coordenadas. Ela corresponde ao número mínimo de passos em uma grade que permite movimentos diagonais, desde que cada passo tenha o mesmo custo:

$$h(n) = \max(|x_2 - x_1|, |y_2 - y_1|, |z_2 - z_1|) \tag{4}$$

A escolha da heurística depende de ela ser compatível com o custo das arestas. A euclidiana é admissível quando nenhuma aresta custa menos que a distância em linha reta entre seus extremos. A Chebyshev é admissível quando os movimentos

diagonais custam o mesmo que os movimentos retos. Já a Manhattan superestima a distância real quando há movimentos diagonais, o que pode levar a caminhos não ótimos.

Como a heurística define a ordem de exploração, ela altera tanto o número de vértices visitados quanto a qualidade do caminho encontrado. Uma heurística admissível garante o caminho ótimo, enquanto uma heurística que superestima o custo pode explorar menos vértices e, ainda assim, não encontrar o menor caminho.

2.2.3 A*

O A* é um algoritmo de busca que combina o custo acumulado com a estimativa do custo restante. Para cada vértice n, ele avalia

$$f(n) = g(n) + h(n) \tag{5}$$

em que $g(n)$ é o custo do caminho desde o vértice inicial até n e $h(n)$ é a heurística de n até o objetivo. Como no Dijkstra, a busca usa uma fila de prioridade, mas ordena os vértices por $f(n)$, e não apenas por $g(n)$. Se a heurística for admissível, o caminho retornado tem o menor custo possível (Hart; Nilsson; Raphael, 1968).

Figura 10 – Pseudocódigo do algoritmo A*.

```
A*(G, W, s, t, h)
1   para cada v ∈ V faça
2       g_v ← inf
3       p_v ← nil
4   g_s ← 0
5   Q ← {s}
6   S ← ∅
7   enquanto Q ≠ ∅ faça
8       # vértice de menor f(u) = g_u + h(u)
9       u ← pop(Q)
10      se u = t então retorna o caminho até t por p
11      S ← S ∪ {u}
12      para cada vértice v ∈ adj(u) faça
13          se v ∉ S e [g_v > g_u + W_{u,v}] então
14              g_v ← (g_u + W_{u,v})
15              p_v ← u
16              se v ∉ Q então Q ← Q ∪ {v}
```

Fonte: Elaborado pelo autor, com base em Gomes (2022) e Hart; Nilsson; Raphael (1968).

Na Figura 10, as diferenças em relação ao Dijkstra estão em negrito. A inicialização das distâncias é igual à do Dijkstra, mas a fila Q começa apenas com o vértice inicial s, pois a busca só precisa explorar a fronteira a partir dele.

A principal diferença está na escolha do vértice a ser expandido. Enquanto o Dijkstra remove de Q o vértice de menor distância, o A* remove o vértice de menor $f(u)$. Quando $h(n) = 0$ para todo n, as duas escolhas coincidem, e o Dijkstra passa a ser um caso particular do A*.

Outra diferença é o critério de parada. O algoritmo termina quando o vértice removido é o objetivo t, e não quando Q esvazia. Essa parada antecipada pode reduzir o número de vértices explorados. Vértices já finalizados em S não são mais atualizados, e um vértice entra em Q na primeira vez que é alcançado.

2.2.4 Weighted A*

O Weighted A* é uma variante do A* em que a heurística é multiplicada por um fator de ponderação w. A função de avaliação passa a ser:

$$f(n) = g(n) + w \cdot h(n) \tag{6}$$

Com $w = 1$, a busca coincide com o A*. Com $w = 0$, a heurística deixa de influenciar a escolha e o algoritmo se comporta como o Dijkstra. Para $w > 1$, a estimativa passa a pesar mais na escolha, o que tende a reduzir o número de vértices explorados. Em contrapartida, o caminho encontrado pode não ser o de menor custo (Pohl, 1970).

Figura 11 – Pseudocódigo do algoritmo Weighted A*.

```
Weighted A*(G, W, s, t, h, w)
para cada v ∈ V faça
    g_v ← inf
    p_v ← nil
g_s ← 0
Q ← {s}
S ← ∅
enquanto Q ≠ ∅ faça
    # vértice de menor f(u) = g_u + * w * · h(u)
    u ← pop(Q)
    se u = t então retorna o caminho até t por p
    S ← S ∪ {u}
    para cada vértice v ∈ adj(u) faça
        se v ∉ S e [g_v > g_u + W_{u,v}] então
            g_v ← (g_u + W_{u,v})
            p_v ← u
            se v ∉ Q então Q ← Q ∪ {v}
```

Fonte: Elaborado pelo autor, com base em Gomes (2022) e Hart; Nilsson; Raphael (1968).

Na Figura 11, as diferenças em relação ao Dijkstra estão em negrito, e são as mesmas do A*, acrescidas do fator w na seleção do vértice. Como a estrutura

do pseudocódigo é a do A*, a única mudança de comportamento vem da função de avaliação, e por isso o restante da explicação do A* se aplica aqui.

O valor de w define o equilíbrio entre velocidade e otimalidade. Valores próximos de 1 mantêm a garantia do A* quando a heurística é admissível. Valores maiores fazem a busca se aproximar do comportamento de uma busca gulosa, que é o caso tratado a seguir.

2.2.5 Greedy Best-First Search

A Greedy Best-First Search ordena os vértices apenas pela heurística, sem considerar o custo acumulado até eles (Russell; Norvig, 2013). Assim, a função de avaliação é:

$$f(n) = h(n) \tag{7}$$

Essa escolha torna a busca rápida, porque ela tende a seguir diretamente na direção do objetivo. Em contrapartida, como o custo acumulado é ignorado, o caminho encontrado pode ser longo e não é garantidamente o de menor custo. A Greedy Best--First Search corresponde ao caso em que o peso da heurística tende ao infinito no Weighted A*.

Figura 12 – Pseudocódigo da Greedy Best-First Search.

```
Greedy Best-First Search(G, s, t, h)
1   para cada v ∈ V faça
2       p_v ← nil
3   Q ← {s}
4   S ← ∅
5   enquanto Q ≠ ∅ faça
6       # vértice de menor h(u)
7       u ← pop(Q)
8       se u = t então retorna o caminho até t por p
9       S ← S ∪ {u}
10      para cada vértice v ∈ adj(u) faça
11          se v ∉ S e v ∉ Q então
12              p_v ← u
13              Q ← Q ∪ {v}
```

Fonte: Elaborado pelo autor, com base em Gomes (2022) e Hart; Nilsson; Raphael (1968).

Na Figura 12, as diferenças em relação ao Dijkstra estão em negrito. Não há g_v nem relaxação: a fila é ordenada apenas por $h(u)$, e cada vértice é inserido uma única vez, na primeira vez em que é descoberto. Por isso, a inicialização não precisa das distâncias, e a condição de atualização depende apenas de o vértice já ter sido visitado ou estar na fila.

2.3 Geração Procedural de Terrenos

A geração procedural consiste em criar conteúdo automaticamente, a partir de algoritmos e de um conjunto de regras ou parâmetros. Em terrenos 3D, essa abordagem produz paisagens complexas e variadas sem modelagem manual. Como o resultado depende apenas dos parâmetros e de uma semente inicial, a mesma configuração pode ser reproduzida com exatidão.

2.3.1 Mapas de altura

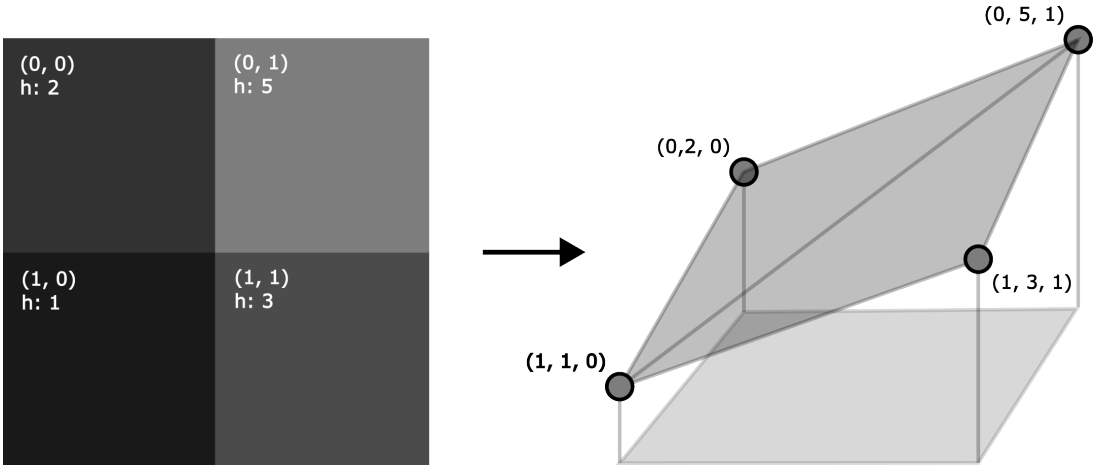
Um mapa de altura é uma matriz bidimensional em que cada elemento armazena a altura do terreno naquele ponto. Quando essa matriz é exibida como imagem em escala de cinza, a intensidade de cada pixel corresponde à elevação. Em geral, os valores são normalizados no intervalo $[0, 1]$ e convertidos em altura por um fator de escala H, que define a altura máxima do terreno.

Para obter a geometria tridimensional, cada elemento (i, j) da matriz é associado a um vértice cujas coordenadas são

$$(x, y, z) = (i, h(i, j), j) \tag{8}$$

em que $h(i, j)$ é a altura armazenada naquele ponto. Dessa forma, o plano horizontal da grade é preservado, e a altura passa a ser a coordenada y (Figura 13).

Figura 13 – Conversão de um mapa de altura em vértices tridimensionais.



Fonte: Elaborado pelo autor.

Dessa forma, um pixel totalmente preto ($v = 0$) resultará na altura mínima, enquanto um pixel totalmente branco ($v = 1$) atingirá a altura máxima estipulada (H), permitindo um controle preciso sobre a amplitude do relevo gerado.

Por fim, são criadas faces conectando cada vértice com seus vizinhos diretos, formando uma malha de triângulos que representa a superfície do terreno. Essa malha pode então ser utilizada como cenário de teste para os algoritmos de *pathfinding*, permitindo avaliar seu desempenho em ambientes tridimensionais complexos. Neste

trabalho, porém, a malha usada pelos grafos de vértices e dual não é obtida dessa forma, e sim pela extração de superfície de um volume de *voxels* (ver Seção 2.4.1).

Figura 14 – Exemplo de malha gerada a partir de um mapa de altura



Fonte: Elaborado pelo autor.

A criação de mapas de altura pode ser feita a partir de imagens reais, como fotografias aéreas ou imagens de satélite, ou gerada proceduralmente por algoritmos de ruído, como o ruído de Perlin (Seção 2.3.2).

2.3.2 Ruído de Perlin

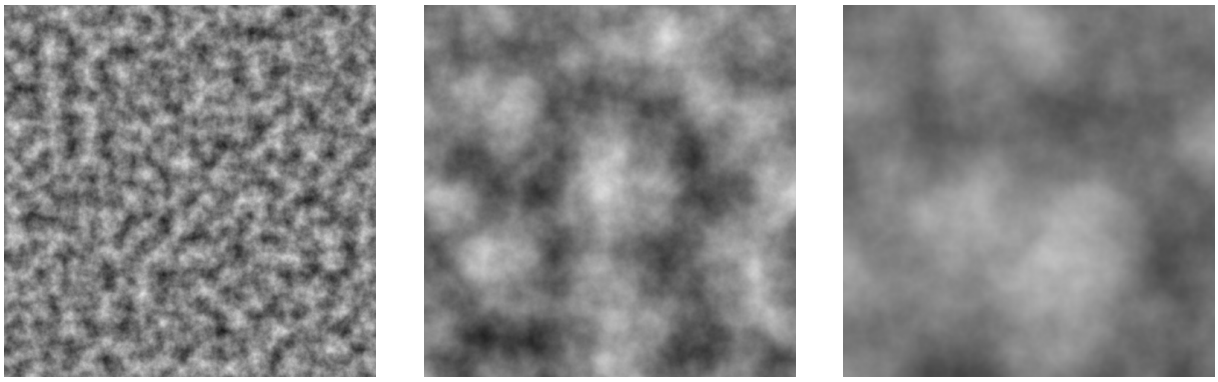
O ruído de Perlin é um algoritmo de geração de ruído gradiente, proposto por Perlin (1985), que produz variações contínuas e suaves. Diferentemente de um ruído puramente aleatório, cada valor depende de uma grade regular de vetores de gradiente, o que faz o resultado variar gradualmente. Por isso, o ruído de Perlin é adequado para simular relevos naturais.

O valor do ruído em um ponto é obtido a partir da célula da grade que o contém. Cada canto c dessa célula possui um vetor de gradiente unitário $\vec{g}_c$, com direção aleatória. Para cada canto, calcula-se o vetor de distância $\vec{d}_c$ entre o ponto e o canto, e o produto escalar entre os dois vetores:

$$s_c = \vec{d}_c \cdot \vec{g}_c = d_{c,x} g_{c,x} + d_{c,y} g_{c,y} \tag{9}$$

Esses valores s_c são as contribuições de cada canto. O tamanho da célula determina o nível de detalhe: células maiores produzem um ruído mais suave, e células menores produzem um ruído mais detalhado.

Figura 15 – Ruído de Perlin com células de diferentes tamanhos.



(a) Ruído gerado com células pequenas.

(b) Ruído gerado com células medianas.

(c) Ruído gerado com células grandes.

Fonte: Elaborado pelo autor.

2.3.2.1 Função de suavização

As contribuições dos cantos são combinadas por interpolação, dentro da própria célula. A interpolação linear simples produz transições com mudança abrupta de inclinação nas bordas da célula, o que se torna visível no terreno. Para evitar isso, o peso de interpolação w é substituído por uma função de suavização $S(w)$. A proposta original de Perlin utiliza uma curva cúbica, e versões posteriores empregam a curva quíntica:

$$S(w) = 6w^5 - 15w^4 + 10w^3 \tag{10}$$

Com essa função, a interpolação entre dois valores a_0 e a_1 é dada por:

$$f(a_0, a_1, w) = a_0 + (a_1 - a_0) \cdot S(w) \tag{11}$$

2.3.2.2 Oitavas

Uma única oitava do ruído tem um nível de detalhe fixo. Para obter relevos com variações em diferentes escalas, somam-se várias oitavas, cada uma com frequência e amplitude próprias. Para a oitava i, com $i = 0, 1, ..., N-1$, tem-se

$$f_i = f \cdot l^i, \quad a_i = a_0 \cdot p^i \tag{12}$$

e o valor final do ruído é a soma das oitavas:

$$V(x, y) = \sum_{i=0}^{N-1} a_i \cdot \text{ruído}(x \cdot f_i, y \cdot f_i) \tag{13}$$

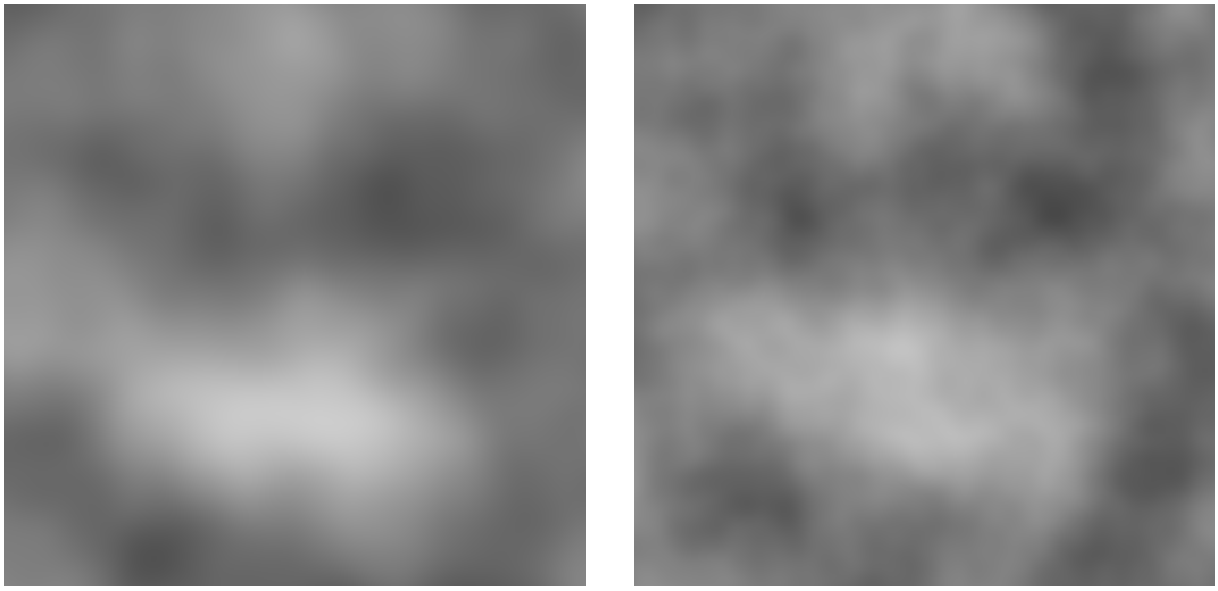
Os parâmetros f, a_0, l, p e N controlam o resultado final de maneiras distintas, descritas a seguir.

A frequência inicial f define a escala do ruído na primeira oitava. Quando f aumenta, o padrão se repete com mais proximidade, e uma mesma área apresenta mais elevações e depressões. Como todas as oitavas são multiplicadas pela mesma proporção de f, a escala de todo o terreno muda, e não apenas um detalhe.

A amplitude inicial a_0 define o alcance vertical da primeira oitava. Como $a_i = a_0 \cdot p^i$, aumentar a_0 multiplica a contribuição de todas as oitavas pelo mesmo fator. Assim, a_0 altera a altura total do relevo sem mudar a proporção entre detalhes grandes e pequenos.

A lacunaridade l determina quanto a frequência cresce de uma oitava para a seguinte. Valores altos fazem as oitavas finais ter frequências muito maiores que as iniciais, de modo que o detalhe se concentra em escalas pequenas e o relevo fica mais irregular. Valores próximos de 1 fazem todas as oitavas ter frequência semelhante, e o resultado se aproxima de um único ruído escalado. A Figura 16 compara ruídos com lacunaridade baixa e alta, mantendo os demais parâmetros.

Figura 16 – Variação da lacunaridade: ruídos com lacunaridade baixa e alta.

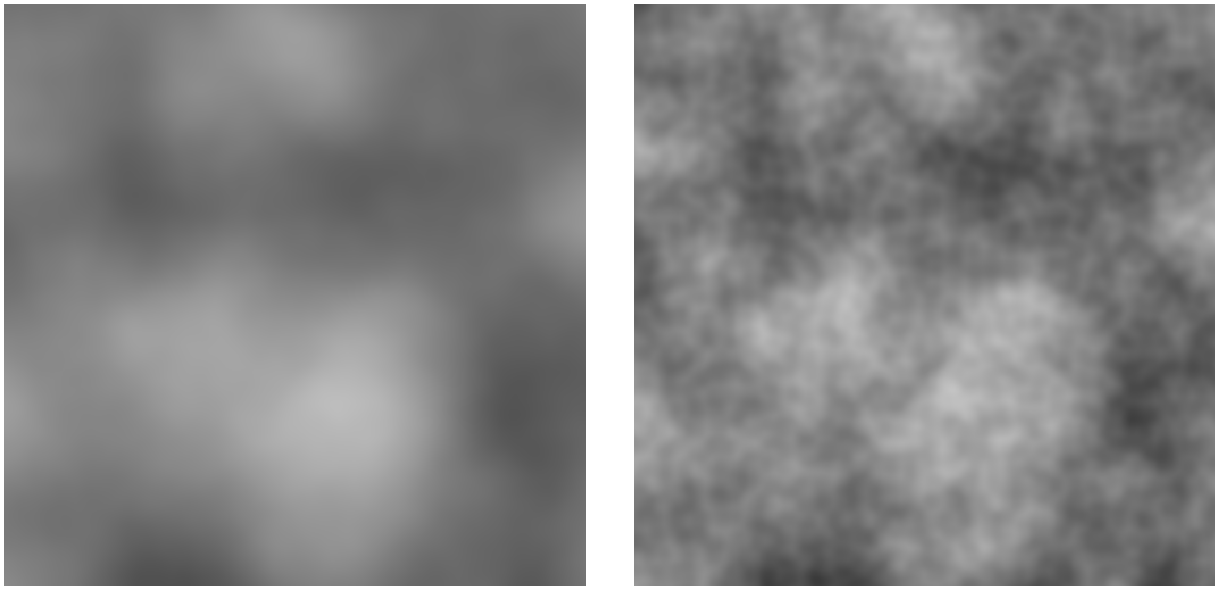


(a) Ruído de baixa lacunaridade. (b) Ruído de alta lacunaridade.

Fonte: Elaborado pelo autor.

A persistência p determina quanto a amplitude diminui de uma oitava para a seguinte. Valores baixos fazem os detalhes finos ter pouca influência, e o relevo fica dominado pelas formas amplas. Valores altos mantêm as oitavas finas com peso comparável ao das oitavas grandes, e o terreno fica mais áspero. A Figura 17 mostra essa diferença, mantendo os demais parâmetros.

Figura 17 – Variação da persistência: ruídos com persistência baixa e alta.

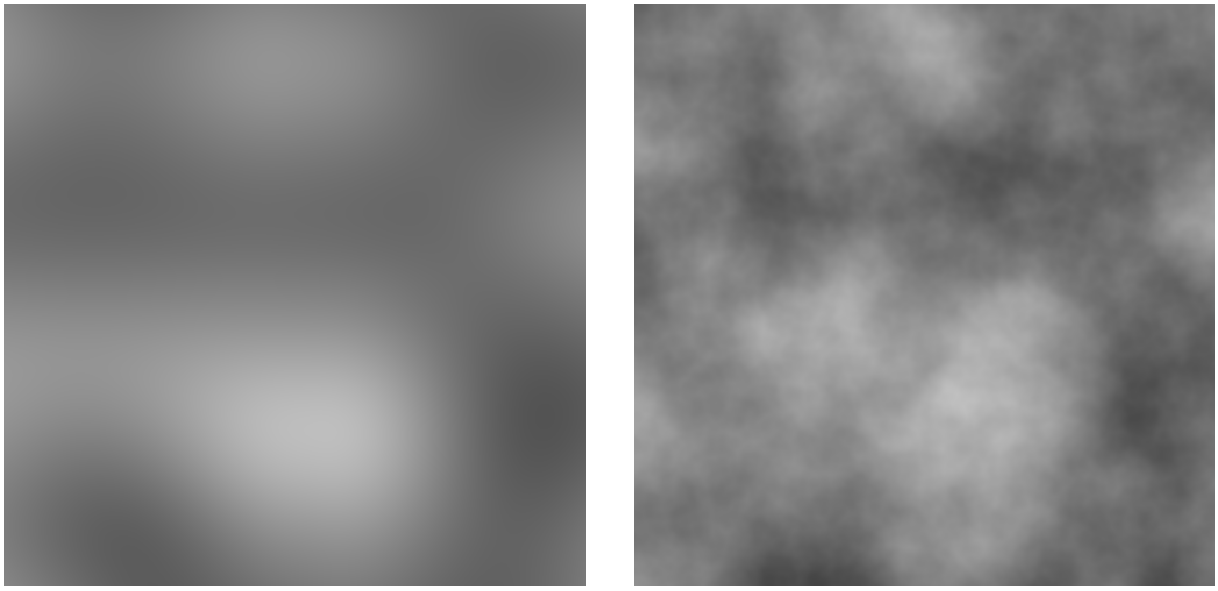


(a) Ruído de baixa amplitude/persistência. (b) Ruído de alta amplitude/persistência.

Fonte: Elaborado pelo autor.

O número de oitavas N define quantas camadas são somadas. Com poucas oitavas, o ruído contém apenas as escalas mais grosseiras. Cada oitava adicional acrescenta detalhes menores, cujo peso é reduzido por p^i. Por isso, com persistência baixa, oitavas adicionais quase não alteram o resultado, e com persistência alta, cada uma contribui de forma perceptível (Figura 18).

Figura 18 – Variação do número de oitavas: ruídos com 1 e 6 oitavas.



(a) Ruído com 1 oitava. (b) Ruído com 6 oitavas.

Fonte: Elaborado pelo autor.

Valores comumente usados são próximos de 2 para a lacunaridade e de 0,5 para a persistência, de modo que cada oitava tem o dobro da frequência e metade da amplitude da anterior. Essa combinação é descrita por Patel (2015), e produz terrenos com grandes elevações e pequenas variações ao mesmo tempo.

2.4 Representações Geométricas de Terrenos

Esta seção descreve formas de representar a geometria visual de terrenos em um ambiente 3D. Cada técnica define como o volume ou a superfície do terreno é armazenado, e essa escolha determina quais formas podem ser descritas.

2.4.1 Voxels

O *voxel* é a unidade cúbica que divide o espaço tridimensional em uma grade regular, de modo análogo ao pixel em imagens bidimensionais. Em uma representação volumétrica de terreno, cada *voxel* pode indicar se a posição correspondente está preenchida por material ou vazia. Essa abordagem descreve cavidades e estruturas sobrepostas, que um mapa de altura não armazena, já que ele guarda apenas uma altura para cada posição horizontal. Neste trabalho, porém, o volume é obtido a partir do próprio mapa de altura: cada coluna é preenchida até a sua altura, arredondada para um número inteiro de *voxels*. Por isso, o volume não contém cavidades nem estruturas sobrepostas, e essa capacidade da representação volumétrica não é explorada.

2.4.2 Marching Cubes

A extração de uma superfície a partir de um campo volumétrico pode ser feita pelo algoritmo de Marching Cubes, proposto por Lorensen; Cline (1987). Para cada

célula do volume, o algoritmo compara os valores dos oito cantos com um limiar, classifica cada canto como acima ou abaixo dele e gera os triângulos correspondentes à configuração resultante. Como essa classificação produz 256 configurações possíveis, o algoritmo usa tabelas de casos para determinar os triângulos de cada uma.

O Transvoxel, apresentado por Lengyel (2010), estende essa abordagem a terrenos baseados em voxels em tempo real. Ele acrescenta tabelas de casos para células de transição entre níveis de detalhe distintos, o que evita rachaduras na superfície nas fronteiras entre blocos de resoluções diferentes. Neste trabalho, o volume é extraído em uma única resolução, e por isso são usadas apenas as tabelas de células regulares do Transvoxel, que correspondem aos casos do Marching Cubes. As células de transição não são utilizadas.

Figura 19 – Exemplo de reconstrução de superfície com Marching Cubes.

Fonte: Elaborado pelo autor.

2.5 Representações Navegáveis de Terrenos

A busca de caminhos depende de uma estrutura que represente as regiões por onde um agente pode se deslocar. Essa estrutura é uma representação navegável, em geral um grafo $G = (V, A)$ (ver Seção 2.1), obtido a partir da geometria do terreno. Seus vértices correspondem a posições ou regiões transitáveis, e suas arestas indicam a passagem entre elas. Diferentes representações partem do mesmo terreno, mas produzem grafos com quantidades distintas de vértices e arestas. Conforme Algfoor; Sunar; Kolivand (2015), o *pathfinding* pode ser dividido em duas etapas: a geração do grafo e o algoritmo de busca.

2.5.1 Grade 2.5D

A grade 2.5D é uma matriz bidimensional de alturas, em que cada célula corresponde a uma posição do plano horizontal e armazena um único valor de altura. Como grafo, ela corresponde ao mapa de altura (ver Seção 2.3.1): cada célula é um vértice, ligado às células vizinhas (Sturtevant, 2012). Neste trabalho, todas as ligações entre vizinhos são mantidas, e a diferença de altura entre as células afeta apenas o peso das arestas. Por armazenar uma altura por posição, a grade descreve a superfície do terreno, mas não representa cavidades nem estruturas sobrepostas.

Figura 20 – Exemplo de grade 2.5D.

Fonte: Elaborado pelo autor.

2.5.2 Grade Voxelizada

A grade voxelizada é uma grade 2.5D em que a altura de cada coluna é arredondada para um número inteiro de *voxels* (ver Seção 2.4.1). Cada coluna do mapa de altura corresponde a um vértice, posicionado no topo da coluna, e as arestas ligam apenas as colunas vizinhas nas quatro direções do plano horizontal. Como a grade 2.5D, ela armazena uma única altura por posição e não representa cavidades nem regiões sobrepostas. As diferenças em relação à grade 2.5D são a altura discretizada e a vizinhança de quatro direções.

Figura 21 – Exemplo de grade voxelizada.

Fonte: Elaborado pelo autor.

2.5.3 Grafo de Vértices

No grafo de vértices, cada vértice da malha poligonal do terreno dá origem a um vértice navegável, e cada aresta liga dois vértices que são extremidades de uma mesma aresta da malha. A estrutura reaproveita a topologia da malha, sem criar vértices novos.

Figura 22 – Exemplo de grafo de vértices.

Fonte: Elaborado pelo autor.

2.5.4 Grafo Dual

No grafo dual, cada polígono da malha corresponde a um vértice, que representa uma região navegável, e dois vértices são ligados por uma aresta quando os polígonos correspondentes compartilham uma aresta da malha. O número de vértices do grafo é, portanto, o número de polígonos da malha, e não o número de seus vértices. Essa construção é a base das malhas de navegação (*navmesh*), nas quais os polígonos são regiões caminháveis e a adjacência entre eles forma o grafo, conforme Snook (2000) e Gurung (2019). A mesma ideia de adjacência entre triângulos é empregada na busca em triangulações, como em Demyen; Buro (2006).

Figura 23 – Exemplo de grafo dual.

Fonte: Elaborado pelo autor.

2.6 Avaliação de desempenho e estatísticas

Esta seção apresenta as ferramentas estatísticas usadas para comparar medições de desempenho obtidas em condições diferentes. Medições como o tempo de execução variam de uma execução para outra, de modo que a diferença observada entre as médias de dois grupos pode resultar dessa variação, e não de uma diferença real entre os métodos comparados. Testes de hipótese permitem avaliar se a diferença observada é compatível com essa variação aleatória. Ferramentas desse tipo são usuais na análise de desempenho de sistemas, como discute Jain (1991).

2.6.1 ANOVA

A ANOVA compara as médias de uma variável resposta entre dois ou mais grupos definidos por um fator. Sua hipótese nula afirma que todas as médias populacionais são iguais, e a hipótese alternativa afirma que ao menos uma delas difere das demais. A estatística do teste é a razão entre o quadrado médio (MQ) entre os grupos e o quadrado médio dentro dos grupos:

$$F = \frac{\mathrm{MQ}_{\mathrm{entre}}}{\mathrm{MQ}_{\mathrm{dentro}}} \tag{14}$$

Valores grandes de F indicam que a variação entre os grupos é grande em relação à variação interna, o que leva à rejeição da hipótese nula ao nível de significância adotado. A validade do teste depende de três pressupostos: independência das observações, normalidade dos resíduos e homocedasticidade, isto é, variâncias iguais entre os grupos (Domingues, 2026). A violação desses pressupostos pode alterar o nível de significância real do teste.

2.6.2 Teste post hoc de Tukey

Quando a ANOVA rejeita a hipótese nula, ela indica que ao menos um par de médias difere, mas não identifica quais pares são. Testes *post hoc* realizam essas comparações após a ANOVA. O teste de Tukey, ou diferença honestamente significativa (HSD, do inglês *honestly significant difference*), compara todos os pares de médias. Ele controla a taxa de erro da família de comparações, de modo que o número de pares comparados não aumente indevidamente a chance de declarar uma diferença falsa (Domingues, 2026).

Para grupos com n observações cada, a menor diferença significativa entre duas médias é dada por:

$$\mathrm{HSD} = q_{\alpha,k,\nu}\sqrt{\frac{\mathrm{MQ}_{\mathrm{dentro}}}{n}} \tag{15}$$

em que q é a estatística da amplitude studentizada, k é o número de grupos, ν são os graus de liberdade do quadrado médio dentro dos grupos e α é o nível de significância. Dois grupos são considerados diferentes quando a diferença absoluta entre suas médias excede o HSD.

2.7 Trabalhos relacionados

2.7.1 Johansson

2.7.2 Kapi

2.7.3 Matriz de diferença

figura: matrix de diferença

2.8 Pipeline

Cada execução do experimento segue a mesma sequência de etapas. A partir de uma semente, gera-se um mapa de altura por ruído de Perlin (ver Seção 2.3.2). Quando o modelo de representação exige geometria, o mapa é convertido em um volume de *voxels*, com cada coluna preenchida até a sua altura arredondada, e esse volume é convertido em uma malha triangular por extração de superfície. Em seguida, o modelo constrói o grafo navegável, os pares de consulta são associados aos vértices mais próximos do grafo, e os algoritmos de busca são executados sobre ele. As métricas de cada execução são registradas e, ao final, analisadas estatisticamente (ver Seção 2.6).

Figura 24 – Etapas do pipeline experimental.

Fonte: Elaborado pelo autor.

2.9 IFCG e visualizador concorrente

O IFCG é uma biblioteca de motor gráfico escrita em C++ sobre OpenGL, desenvolvida pelo autor e inicialmente vinculada à disciplina de Computação Gráfica do IFC Campus Blumenau[1]. Neste trabalho, ela é a camada de visualização: exibe o terreno gerado, a malha de navegação e os caminhos encontrados pelos algoritmos de busca, permitindo a inspeção visual dos resultados.

2.9.1 Arquitetura e *Singleton*

O núcleo da biblioteca é a classe `Engine`, que reúne a janela, o contexto, o renderizador, o tratamento de entrada e o gerenciador de tarefas. A aplicação mantém

[1] Disponível em: https://github.com/andrevbastos/ifcg

uma única janela e um único contexto, e por isso a classe segue o padrão *Singleton* (Gamma et al., 1994): o construtor é privado, e a instância é criada por `Engine::init` e destruída por `Engine::terminate`.

2.9.2 Janela e contexto

A classe `Window` encapsula a criação da janela pela biblioteca GLFW e expõe suas dimensões e o identificador da janela. A classe `Context` inicializa a GLFW, carrega as funções do OpenGL com a biblioteca GLAD, processa os eventos da janela e informa o tempo decorrido desde a inicialização.

2.9.3 Renderização e laço principal

A classe `Renderer` mantém a fila de malhas desenhadas a cada quadro, além do *shader* ativo, da câmera e dos planos de recorte, cujos valores padrão são campo de visão de 90 graus e planos próximo e distante em 0,1 e 100 unidades. O laço principal é executado por `Engine::loop`, que recebe um `LoopConfig` com funções de callback fornecidas pela aplicação.

Em cada iteração, as tarefas pendentes destinadas à thread principal são executadas antes da leitura de entrada e novamente antes do corpo do laço. Em seguida, ocorrem a atualização da câmera, a renderização da fila e a troca de buffers. Quando há limite de quadros por segundo, a thread aguarda o tempo restante do quadro.

2.9.4 Malhas e composição

`MeshBase` define a interface comum das malhas, com a matriz de modelo e as transformações de translação, escala, rotação e reflexão. `Mesh` armazena uma malha indexada, gerencia os buffers de vértices e de índices e a desenha por meio de `glDrawElements`. `MeshTree` implementa o padrão *Composite* (Gamma et al., 1994): seus filhos são malhas simples ou compostas, e o desenho é repassado a cada filho com a matriz de modelo acumulada do nó pai. Dessa forma, uma malha simples e uma composta são tratadas pela mesma interface.

Figura 25 – Diagrama de classes do padrão *Composite* nas malhas do IFCG.

Fonte: Elaborado pelo autor.

2.9.5 Shaders e buffers

`Shader` compila e vincula os programas de sombreamento escritos em GLSL (do inglês *OpenGL Shading Language*). As classes `VBO` (do inglês *vertex buffer object*), `VAO` (do inglês *vertex array object*) e `EBO` (do inglês *element buffer object*) encapsulam, respectivamente, os buffers de vértices, a descrição dos atributos de vértice e os

buffers de índices. Essa organização segue o pipeline de renderização descrito em Vries (2014).

2.9.6 Câmera e entrada

`Camera` atualiza as matrizes de visão e de projeção a partir do campo de visão e dos planos de recorte, e recebe a movimentação pela janela da GLFW. `Keys` associa funções de callback a teclas e ações definidas pela aplicação.

2.9.7 Concorrência e *TaskMaster*

O contexto do OpenGL está associado à thread que o ativa, normalmente a principal, e as chamadas à API devem partir dela. O IFCG separa, portanto, o trabalho computacional, que pode ser executado em paralelo, da atualização da unidade de processamento gráfico (GPU, do inglês *graphics processing unit*), que permanece na thread principal.

O *TaskMaster* implementa esse trabalho paralelo como um *pool* de threads. O número de threads trabalhadoras é uma a menos que o de unidades de processamento disponíveis, mantendo-se pelo menos uma. As tarefas entram em três filas, de prioridade alta, média e baixa, e cada trabalhadora retira a próxima tarefa da fila de maior prioridade que não esteja vazia (Ladeira, 2026). A sincronização usa um `mutex` e uma variável de condição, e as threads são `std::jthread`, que recebem um `std::stop_token` para encerramento cooperativo Williams (2019).

Figura 26 – Pool de threads e filas de prioridade do *TaskMaster*.

Fonte: Elaborado pelo autor.

`Engine` oferece três operações sobre o *TaskMaster*. `runAsync` envia uma tarefa ao *pool*. `runOnMainThread` enfileira uma função para execução na thread principal, na próxima iteração do laço. `runAsyncThenMain` combina as duas: a tarefa é calculada em uma trabalhadora, e o resultado é entregue a uma função que roda na thread principal. No visualizador, a geração do ruído e da geometria do terreno e a construção do grafo de navegação ocorrem nas trabalhadoras, enquanto a criação das malhas e o envio à GPU ficam na thread principal.

Figura 27 – Cena renderizada pelo visualizador do TCC.

Fonte: Elaborado pelo autor.

2.10 lwGraph<T>

O grafo usado nos experimentos é representado pela classe `lwGraph<T>`, uma estrutura genérica em que cada vértice armazena um dado do tipo `T`. A biblioteca Graph, desenvolvida pelo autor, define a versão base `common::lwGraph<T>` e a especialização `undirected::lwGraph<T>`, usada neste trabalho, que trata as arestas como não direcionadas.

Os vértices são armazenados em um vetor contíguo (`std::vector<T>`), e a lista de adjacência é um vetor de vetores de arestas (`std::vector<std::vector<lwEdge>>`). Cada `lwEdge` guarda apenas o índice do vértice de destino e o peso, sem objetos individuais para nós ou arestas. Essa disposição mantém os dados em blocos contíguos, o que favorece a localidade de memória quando a busca percorre os vizinhos de um vértice.

Na especialização não direcionada, `addEdge` insere a aresta nas listas dos dois vértices, de modo que ela vale nos dois sentidos, como descrito em Seção 2.1. Depois de construído, o grafo não é modificado pelos algoritmos de busca, que o recebem por referência constante. Por isso, uma mesma instância pode ser lida por mais de uma thread, sem sincronização adicional.

Para o cálculo de memória, a função `graphMemoryBytes` soma o tamanho do objeto do grafo, os dados dos vértices, os vetores de adjacência e a capacidade alocada para as arestas. Usa-se a capacidade, e não o tamanho, porque ela inclui a folga reservada pelo crescimento dos vetores, que também ocupa memória.

Figura 28 – Representação do `lwGraph`: vetor de vértices e vetores de adjacência.

Fonte: Elaborado pelo autor.

2.11 search.hpp

Os algoritmos de busca estão no arquivo `search.hpp`, no *namespace* `search`. Cada um deles é uma função genérica sobre `common::lwGraph<T>`, que recebe o vértice inicial, o vértice objetivo e, quando necessária, uma heurística. Todos retornam uma estrutura `SearchResult`, com o caminho encontrado, o custo acumulado e os contadores de execução.

Os quatro algoritmos têm implementações separadas, e Dijkstra não é tratado como A* com heurística nula. Todos seguem a mesma estrutura de busca ponto a ponto: a lista aberta começa apenas com o vértice inicial, e a busca termina quando o vértice objetivo é retirado dela. No Dijkstra, isso difere do pseudocódigo da Figura 8, que insere todos os vértices em Q e só termina quando a fila esvazia. Como os vértices são retirados em ordem de custo, o caminho até o objetivo é o mesmo nas duas formas.

2.11.1 Algoritmos

Dijkstra ordena a lista aberta pelo custo acumulado `g`. O A* ordena pela soma `f = g + h`, usando como heurística a distância euclidiana tridimensional entre o vértice e o objetivo. O Weighted A* usa `f = g + w·h`, com o fator `w` igual a 1,5 nos experimentos. A Greedy Best-First Search (GBFS) ordena somente pela heurística.

Na GBFS, como o custo acumulado não influencia a ordem, cada vértice entra na lista aberta uma única vez, na primeira vez em que é descoberto. Diferentemente do pseudocódigo da Figura 12, a implementação mantém o custo acumulado de cada vértice: um custo menor encontrado depois, enquanto o vértice ainda está na lista aberta, atualiza o pai do vértice e o custo registrado, sem reinseri-lo. Por isso, o caminho retornado pode ter custo menor que o da GBFS clássica.

A lista aberta é uma fila de prioridade sem operação de redução de chave. Nos demais algoritmos, quando um vértice que já está na lista recebe um custo menor, ele é inserido novamente com a nova prioridade, e as entradas antigas são descartadas quando retiradas da fila, se o vértice já tiver sido expandido. No A* e no Weighted A*, os empates de prioridade são desfeitos pelo menor valor da heurística e, em seguida, pelo menor índice do vértice. No Dijkstra, que não usa heurística, e na GBFS, cuja prioridade já é a própria heurística, o desempate é feito apenas pelo índice.

2.11.2 Aquisição de dados

Os contadores são incrementados dentro dos próprios algoritmos. `expanded` registra os vértices retirados da lista aberta e processados, `generated` registra as inserções na lista aberta, incluindo as reinserções de um mesmo vértice, e `peakOpen` registra o maior tamanho que a lista atinge durante a busca, contando também as entradas antigas ainda não descartadas. Por fazerem parte da execução, esses valores são os mesmos em toda repetição com o mesmo grafo e os mesmos vértices.

O tempo de execução é medido com `std::chrono::steady_clock` ao redor da chamada do algoritmo, sem incluir a construção do grafo nem a geração do terreno. Quando uma busca leva menos de 5 ms, a resolução do relógio e a variação entre chamadas pesam sobre a medição. Nesse caso, a busca é repetida até que o tempo somado ultrapasse 5 ms, com no máximo 100 000 repetições, e registra-se a mediana dos tempos e o número de repetições (Lilja, 2000). Buscas mais longas são medidas uma única vez. Separadamente, mede-se o tempo de inicialização dos vetores de estado da busca para o número de vértices do grafo, já que essa inicialização faz parte de cada chamada. Em seguida, calcula-se o comprimento geométrico do caminho sobre a superfície. Cada segmento é amostrado em subpassos de no máximo 0,25 unidade horizontal, com a altura obtida por interpolação bilinear do mapa de altura. Esse valor independe dos pesos definidos no grafo, ao contrário do custo acumulado.

Os pontos de início e de fim de cada par de consulta são associados ao vértice mais próximo do grafo por busca linear, e a distância de ajuste é emitida no registro de diagnóstico da execução. Se algum algoritmo não encontrar caminho, a aquisição

é interrompida com erro, pois a ausência de caminho indicaria falha na construção do grafo.

2.12 Gerador de terrenos

O gerador produz o mapa de altura de cada terreno a partir de um conjunto de parâmetros e de uma semente. Sua implementação está no *namespace* `perlin2D`, e os parâmetros ficam reunidos na estrutura `NoiseConfig`: largura e altura do mapa, comprimento de onda base (`wave`), frequência inicial (`freq`), lacunaridade (`lac`), amplitude (`amp`), persistência (`pers`), expoente (`exp`), semente (`seed`) e número de oitavas (`octaves`).

Os gradientes de cada ponto da grade são vetores unitários cuja direção é obtida por uma função de embaralhamento (*hash*) sobre as coordenadas inteiras e a semente. Assim, o mesmo par de coordenadas e semente sempre produz o mesmo gradiente, sem que seja necessário armazenar uma tabela de gradientes (Zipped, 2023). Os valores entre os cantos de cada célula são combinados pela função quíntica de suavização (ver Seção 2.3.2.1).

Para cada oitava i, o gerador sorteia um deslocamento de 0 a 1000 em cada eixo, a partir de um gerador de números aleatórios inicializado com a semente, e usa a semente derivada $\mathrm{semente} + 1013i$ para os gradientes. A frequência de cada oitava cresce pela lacunaridade, e a amplitude decresce pela persistência, como descrito em Seção 2.3.2.2.

A soma das oitavas é dividida pela raiz quadrada da soma dos quadrados das amplitudes. Essa normalização impede que a persistência, isoladamente, altere a amplitude do relevo. Em seguida, o valor é multiplicado pelo fator `amp`, limitado ao intervalo $[-1, 1]$, reescalado para $[0, 1]$ e elevado ao expoente `exp`. Cada valor v desse mapa é convertido em uma altura $y = 1 + v(H - 1)$, em que H é a escala vertical do terreno.

A função `generateOctaveMaps` produz um mapa por oitava, normalizado isoladamente, e `saveNoiseAsPNG` grava qualquer mapa em escala de cinza.

Figura 29 – Diferentes representações navegáveis do mesmo terreno.

Fonte: Elaborado pelo autor.

2.13 GraphModel e GraphFactory

Cada representação navegável é descrita por uma estrutura `GraphModel`, com três campos: o nome usado na linha de comando e nos arquivos de saída (`name`), a indicação de exigência de geometria (`requiresGeometry`) e a fábrica do grafo (`create`). A fábrica é uma `std::function` que recebe um `GraphBuildInput` e devolve um `NavigationGraph`, alias de `undirected::lwGraph<Vertex3D>`. O `GraphBuildInput` reúne

referências ao mapa de altura e à malha extraída, além da largura, da escala vertical e da profundidade do terreno.

O campo `requiresGeometry` indica se a fábrica consome a malha. O laço experimental usa essa indicação para decidir se extrai a geometria antes de construir o grafo. Os quatro modelos registrados são:

- `2_5d` com `createGrid2_5D`, que lê apenas o mapa de altura;
- `voxel` com `createVoxelGraph`, que lê apenas o mapa de altura;
- `vertex` com `createVertexToVertex`, que parte da malha;
- `dual` com `createPolygonToPolygon`, que também parte da malha.

A `createGrid2_5D` cria um vértice por pixel do mapa de altura, posicionado na altura $y = 1 + v(H - 1)$. Cada célula se liga aos vizinhos a leste, ao sul, a sudeste e a sudoeste, de modo que a vizinhança é de oito direções. O peso de cada aresta é a distância euclidiana tridimensional entre os vértices.

A `createVoxelGraph` constrói a grade voxelizada. Ela cria um vértice por pixel do mapa de altura, isto é, por coluna, com a altura arredondada para um número inteiro de voxels, sem construir o volume. Cada vértice se liga apenas aos vizinhos a leste e ao sul, ou seja, a vizinhança é de quatro direções sobre a superfície. O peso também é a distância euclidiana tridimensional, o que faz a diferença de altura entre colunas contribuir para o custo.

Os grafos de vértices e dual partem da malha extraída por Marching Cubes de um volume preenchido até a altura arredondada de cada coluna. Assim, as três representações que não são a grade 2.5D usam alturas discretizadas.

A `createVertexToVertex` cria um vértice para cada vértice da malha. Para cada triângulo, conecta seus três vértices por arestas, e uma aresta repetida entre triângulos vizinhos é inserida uma única vez.

A `createPolygonToPolygon` cria um vértice para cada triângulo, posicionado no seu baricentro. Duas faces que compartilham uma aresta da malha são ligadas por uma aresta cujo peso é a distância entre os baricentros. Essa fábrica supõe que vértices compartilhados entre triângulos tenham exatamente as mesmas coordenadas na malha.

Nenhuma fábrica aplica critério de transitabilidade. Todas as conexões geradas são mantidas, e a diferença de altura entre os vértices afeta apenas o peso das arestas.

2.14 Instrumentação

A instrumentação reúne os mecanismos que registram as medições durante os testes: o procedimento que gera os terrenos e executa as buscas, a classe que acumula e exporta os resultados, o script que repete as execuções em condições controladas e a interface de linha de comando que dá acesso a esses recursos.

2.14.1 Procedimento dos testes

As quatro variáveis são avaliadas por funções próprias, `testScale`, `testLacunarity`, `testPersistence` e `testFrequency`, que seguem o mesmo procedimento. Cada teste executa um número de passos. Em cada passo, são gerados terrenos com a configuração correspondente à variável testada. Cada terreno recebe uma semente sorteada a partir da semente base da execução, e seu grafo é construído pela representação selecionada. Em seguida, são usados dois pares de consulta sobre o terreno, e cada algoritmo é executado sobre cada par. Os resultados de um terreno só são registrados depois que todos os pares e algoritmos foram executados.

Na escala, a dimensão do terreno cresce a cada passo, e a altura e o comprimento de onda acompanham a dimensão, de modo que apenas a resolução muda entre os passos. A escala, portanto, não é um parâmetro do ruído, e sim a resolução da grade sobre a qual o mesmo relevo é amostrado. Na lacunaridade e na persistência, a dimensão e o comprimento de onda são fixos, e o parâmetro testado cresce linearmente do primeiro ao último passo. No comprimento de onda, a dimensão é fixa, e o comprimento de onda base assume uma lista fixa de valores. Como a frequência do ruído é o inverso do comprimento de onda, esse teste avalia a frequência do relevo. Todos os testes usam seis oitavas.

Se o grafo resultante não tiver vértices, o terreno é refeito com outra semente. Se algum algoritmo não encontrar caminho entre os pares, a execução é interrompida com erro, pois isso indicaria falha na construção do grafo.

2.14.2 Statistics

A classe `Statistics` acumula em memória os registros de cada execução, protegidos por um mutex, o que permite registrar resultados a partir de mais de uma thread. Cada registro, do tipo `Results`, reúne:

- identificação: algoritmo, valor da variável testada, semente do terreno e índice do par de consulta;
- grafo: tempo de construção, número de vértices e de arestas, memória estrutural e, nos grafos de vértices e dual, tempo de extração da malha;
- busca: custo do caminho, tempo de execução, tempo de busca com repetições e número de repetições, número de vértices do caminho e comprimento geométrico;
- contadores: vértices expandidos, vértices gerados, pico da lista aberta e chamadas da heurística;
- terreno: número de máximos locais, fração de células elevadas, desvio padrão da altura e declividade média;
- consulta: distância em linha reta entre início e destino, razão entre a heurística no início e o custo ótimo, subida acumulada e declividade máxima do caminho;
- inicialização: tempo de inicialização dos vetores de estado da busca.

As métricas de terreno e de consulta são calculadas depois das buscas, fora dos intervalos cronometrados. A fração de células elevadas conta as células com altura

de pelo menos um desvio padrão acima da média, e os máximos locais são contados entre essas células, com vizinhança de oito direções.

A exportação grava um arquivo CSV, com cabeçalho e uma linha por registro. Os campos opcionais ficam vazios quando não se aplicam ao registro. O nome da coluna da variável testada é informado pelo próprio teste: Escala, Lacunaridade, Persistência ou Comprimento de Onda. Cada linha também traz o estado da máquina no início e no fim da execução, com a frequência do núcleo usado e a carga média do sistema, e o pico de memória do processo, que são os mesmos para todos os registros de um arquivo.

2.14.3 coleta_completa.sh

O script `coleta_completa.sh` conduz a campanha de coleta. Ele compila o executável em modo *Release* e realiza cinco execuções. Em cada execução, roda um bloco para cada combinação de variável testada e representação, isto é, 16 blocos, cada um como um processo `tcc stats` separado, com dez repetições por passo e quatro passos. A semente base é fixa, de modo que as representações medem os mesmos terrenos, e a semente de cada execução é a semente base somada ao índice da execução.

A ordem dos blocos dentro de cada execução é embaralhada pelo algoritmo de Fisher-Yates, com um gerador de números pseudoaleatórios inicializado com a semente da execução. Assim, a ordem é reproduzível e muda entre as execuções, o que evita que uma mesma representação seja sempre medida no mesmo momento da campanha, quando o estado térmico da máquina pode ser diferente.

Entre blocos consecutivos de uma mesma sessão, o script faz uma pausa de 60 segundos. Antes de cada bloco, espera até que a temperatura do pacote do processador fique em até 45 °C, por no máximo 15 minutos. Se esse limite for atingido, o bloco é iniciado mesmo assim. O turbo do processador é desligado durante a coleta e restaurado ao final, e a execução é fixada a um núcleo com `taskset`. Antes e depois de cada bloco, o script verifica se o notebook está ligado à tomada, porque a frequência do processador muda na bateria. Se não estiver, a coleta é abortada. Os contadores de *throttling* térmico do núcleo e do pacote são lidos antes e depois de cada bloco. Se houver *throttling*, o bloco é descartado e refeito, em até três tentativas, e na última tentativa o resultado é mantido e sinalizado.

Os arquivos de saída e os registros da saída padrão e de erro de cada bloco ficam em uma pasta própria. Um manifesto registra, para cada tentativa, o horário de início, a duração, o código de saída, os eventos de *throttling*, as temperaturas inicial e final, a posição do bloco na ordem, o *commit* do código, a existência de alterações não registradas no repositório e as opções de compilação. Cada bloco concluído é anotado em um arquivo de controle, o que permite retomar uma campanha interrompida sem repetir os blocos já concluídos. Durante a coleta, a suspensão do sistema é impedida com `systemd-inhibit`.

2.14.4 Interface de linha de comando (CLI)

A interface de linha de comando (CLI, do inglês *command-line interface*) é construída com a biblioteca CLI11 e reúne cinco subcomandos. O subcomando `test` executa uma geração visual com os parâmetros informados, e `heuristics` executa a mesma visualização comparando heurísticas na Greedy Best-First Search. O subcomando `save` grava o mapa de ruído em um arquivo PNG, com a opção de salvar cada oitava separadamente. O subcomando `stats` executa a aquisição de uma variável em uma representação e recebe o número de repetições, o número de passos, a semente base, o diretório de saída e, opcionalmente, os índices dos passos a executar. O subcomando `mcbench` mede o tempo de extração do Marching Cubes para uma lista de sementes. A CLI11 valida os parâmetros e rejeita opções desconhecidas e valores fora dos intervalos permitidos.

3 METODOLOGIA

Este capítulo descreve o método adotado para responder à questão central deste trabalho. Inicialmente, caracteriza-se o tipo de pesquisa. Em seguida, apresentam-se o ambiente experimental, as variáveis e os níveis de teste, o procedimento de coleta e as métricas, o protocolo estatístico utilizado na análise e as ameaças à validade dos resultados.

3.1 Tipo de pesquisa

Este trabalho caracteriza-se como uma pesquisa experimental de natureza quantitativa, com abordagem comparativa e análise estatística. Os experimentos são conduzidos em terrenos gerados proceduralmente, nos quais cada fator de geração é variado isoladamente, mantendo-se os demais fixos (Jain, 1991). Os algoritmos de busca e as representações navegáveis são comparados sobre os mesmos terrenos, e as diferenças entre eles são avaliadas por meio de testes estatísticos sobre as medições coletadas.

3.2 Ambiente

Os experimentos foram executados em um notebook com processador Intel Core i5-1235U, de arquitetura híbrida, com 12 processadores lógicos e 15 GiB de memória. A coleta foi feita em um sistema isolado, iniciado a partir de um pendrive bootável com Arch Linux, kernel 7.2.14-arch1-1. O código foi compilado com o compilador GCC 16.2.1 e com o CMake 4.4.5, em modo Release e padrão C++20.

As dependências são as seguintes: a biblioteca CLI11, na versão 2.7.2, incluída no repositório; a nlohmann/json, na versão 3.11.3, obtida por URL; as bibliotecas glm 1.0.3 e Mesa 26.2.6, instaladas no sistema; e as bibliotecas IFCG e graph_project, repositórios públicos do autor.

Durante a coleta, o governador de frequência era powersave, com o driver intel_pstate, e a busca foi fixada no núcleo 2 com taskset. O turbo do processador permaneceu desligado em todas as coletas, para reduzir a ocorrência de throttling térmico.

3.3 Variáveis

As variáveis independentes são a representação navegável, o algoritmo de busca e os parâmetros de geração do terreno. Os parâmetros de escala, lacunaridade, persistência e comprimento de onda são testados um de cada vez, com os demais mantidos nos valores fixos de cada teste, indicados na Seção 3.4. As métricas de desempenho são o tempo de busca, o número de vértices expandidos e o pico da lista aberta. A métrica de qualidade do caminho é o custo relativo, definido como o custo do caminho dividido pelo custo obtido pelo algoritmo de Dijkstra no mesmo terreno, representação e par de consulta. A Tabela 1 resume essas variáveis.

Tabela 1 – Variáveis independentes e dependentes do experimento.

Tipo	Variável	Descrição
Independente	Representação navegável	Grade 2.5D, grade voxelizada, grafo de vértices e grafo dual da malha
Independente	Algoritmo de busca	Dijkstra, A*, Weighted A* (peso 1,5) e Greedy Best-First Search
Independente	Parâmetro de geração	Escala, lacunaridade, persistência e comprimento de onda (testados separadamente)
Dependente (desempenho)	Tempo de busca	Tempo da busca, excluída a construção do grafo, com mediana de repetições nas buscas abaixo de 5 ms
Dependente (desempenho)	Vértices expandidos	Esforço da busca, medido em vértices expandidos
Dependente (desempenho)	Pico da lista aberta	Maior número de elementos simultâneos na lista aberta
Dependente (qualidade)	Custo relativo	Custo do caminho dividido pelo custo do Dijkstra no mesmo par de consulta

3.4 Níveis

Na escala, a dimensão do terreno assume os valores 500, 600, 700 e 800, com altura igual à dimensão e comprimento de onda igual à metade dela, lacunaridade 2 e persistência 0,5. Na lacunaridade, o valor varia de 1,5 a 2,25 em quatro níveis equidistantes, com dimensão 500, comprimento de onda 250 e persistência 0,5. Na persistência, o valor varia de 0,3 a 0,7 nos mesmos moldes, com lacunaridade 2. No comprimento de onda, o valor base assume 200, 275, 350 e 500, com dimensão 500, lacunaridade 2 e persistência 0,5. Todos os testes usam seis oitavas.

3.5 Procedimento e métricas

O procedimento de coleta é executado separadamente para cada teste (escala, lacunaridade, persistência e comprimento de onda) e para cada representação navegável. Em cada teste, o parâmetro avaliado assume quatro níveis, e para cada nível são avaliados 50 terrenos, distribuídos em cinco execuções independentes com dez repetições cada. Cada bloco, formado por um teste e uma representação, roda em um processo separado em cada execução. A semente de cada execução é derivada de uma semente base fixa, de modo que todas as representações medem os mesmos terrenos.

Para cada terreno, o mapa de altura é gerado por ruído de Perlin com seis oitavas e convertido no grafo da representação avaliada. Em seguida, são definidas as duas diagonais do terreno como consultas de canto a canto. Os pontos de início e de destino são associados ao vértice mais próximo do grafo, e cada algoritmo é executado sobre cada consulta. Um terreno cujo grafo fica vazio é gerado novamente com outra semente. Se algum algoritmo não encontra caminho, a execução é interrompida, pois isso indicaria falha na construção do grafo.

As métricas registradas são: o tempo de busca (ver Seção 2.11); o número de vértices expandidos; o pico da lista aberta; e o custo relativo, calculado como o custo do caminho dividido pelo custo do algoritmo de Dijkstra na mesma consulta. Entre os blocos, há uma pausa de 60 segundos e uma espera até que a temperatura do processador fique em até 45 °C. Antes de cada cronometragem, a mesma busca é executada uma vez sem medição, para que a memória e o cache estejam aquecidos. Blocos com *throttling* térmico são refeitos, em até três tentativas. A ordem dos blocos é embaralhada em cada execução (ver Seção 2.14.3).

A extração da malha pelo algoritmo de Marching Cubes é medida em cada terreno dos grafos de vértices e dual, separadamente do tempo de construção do grafo. Como esses dois grafos usam a mesma malha, esse custo se soma ao tempo de construção de ambos.

3.6 Protocolo estatístico

A unidade experimental é o terreno. Para cada terreno, calcula-se a média dos logaritmos naturais das medidas das duas consultas, e todas as métricas são analisadas nessa escala logarítmica, já que tempos, contagens e custos tendem a variar de forma multiplicativa.

Os dados são analisados por uma ANOVA em modelo de split-plot, conforme Seção 2.6.1. O nível do parâmetro é um fator entre terrenos, pois cada nível tem seus próprios terrenos. O algoritmo e a representação são fatores dentro do terreno, medidos sobre o mesmo terreno. Cada efeito dentro do terreno é testado contra sua interação com o terreno, e os graus de liberdade são corrigidos pelo método de Greenhouse-Geisser em todos os efeitos dentro do terreno, sem teste prévio de esfericidade. O nível de significância é $\alpha = 0{,}05$.

O teste post-hoc de Tukey, descrito em Seção 2.6.2, é aplicado depois da ANOVA de duas formas. Dentro do terreno, comparam-se as representações para cada algoritmo e os algoritmos para cada representação, com o erro da interação. Entre níveis, comparam-se os níveis para cada par de algoritmo e representação, tratados como grupos independentes. O custo relativo é analisado apenas para a Weighted A* e a Greedy Best-First Search, já que o custo relativo do Dijkstra é igual a 1 por definição, e o do A* também é igual a 1, pois o peso de cada aresta é a distância euclidiana entre seus extremos, o que torna a heurística consistente (ver Seção 2.2.2).

A normalidade dos resíduos é verificada pelo teste de Shapiro-Wilk, separadamente dentro do terreno e entre terrenos. Como esse teste rejeita desvios pequenos quando o número de observações é grande, a estatística W é lida junto com o valor de p. As análises são feitas em Python, com as versões listadas na seção de Ambiente.

3.7 Ameaças

Quanto à validade interna, o tempo de busca inclui a inicialização dos vetores de estado da busca, que cresce com o número de vértices e pesa mais, proporcionalmente, em buscas com poucas expansões. Esse tempo de inicialização é medido

separadamente para cada grafo, o que permite estimar a sua parcela no tempo de busca.

Quanto à validade de construção, o peso da Weighted A* foi fixado em $1,5$, de modo que os resultados se referem a esse valor. O custo relativo é comparável dentro de cada representação, pois usa o mesmo grafo como referência, mas não é diretamente comparável entre representações.

Quanto à validade estatística, os resíduos não seguem distribuição normal pelo teste de Shapiro-Wilk, sobretudo dentro do terreno. A transformação logarítmica e o desenho balanceado, com o mesmo número de terrenos por nível, reduzem o efeito dessa violação sobre a ANOVA, mas os valores de p devem ser lidos com cautela.

Quanto à validade externa, os terrenos são gerados apenas por ruído de Perlin, em um único computador e sem obstáculos dinâmicos. Os resultados se referem a esse tipo de terreno e a esse ambiente.

4 RESULTADOS E DISCUSSÃO

5 CONCLUSÃO

REFERÊNCIAS

ALGFOOR, Zeyad Abd; SUNAR, Mohd Shahrizal; KOLIVAND, Hoshang. A Comprehensive Study on Pathfinding Techniques for Robotics and Video Games. **International Journal of Computer Games Technology**, v. 2015, p. 1–11, 2015.

CORMEN, Thomas H. *et al*. **Introduction to Algorithms**. 3. ed. Cambridge, MA: MIT Press, 2009.

DEMYEN, Douglas; BURO, Michael. Efficient Triangulation-Based Pathfinding. *In:* AAAI Press, 2006.

DIJKSTRA, Edsger W. A Note on Two Problems in Connexion with Graphs. **Numerische Mathematik**, v. 1, n. 1, p. 269–271, 1959.

DOMINGUES, Jurandir Junior. **Material didático**. Blumenau, SC, 2026.

GAMMA, Erich *et al*. **Design Patterns: Elements of Reusable Object-Oriented Software**. *[S.l.]:* Addison-Wesley, 1994.

GOMES, Paulo César Rodacki. **Grafos: conceitos fundamentais, algoritmos e aplicações**. Blumenau, SC: Editora do Instituto Federal Catarinense, 2022.

GURUNG, Sabin. **Extension On Navigation Meshes**. Bachelor Thesis—Barcelona: Universitat Politécnica de Catalunya, 2019.

HART, Peter E. ; NILSSON, Nils J. ; RAPHAEL, Bertram. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. **IEEE Transactions on Systems Science and Cybernetics**, v. 4, n. 2, p. 100–107, 1968.

JAIN, Raj. **The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling**. New York: John Wiley & Sons, 1991.

JOHANSSON, E.; OTHERS. **Adapting Pathfinding Algorithms for 3D Environments: Performance Analysis and Real-World Applications**. Master's thesis—*[S.l.: S.n.]*.

KAPI, Azyan Yusra; SUNAR, Mohd Shahrizal; ALGFOOR, Zeyad Abd. A Comparison of Pathfinding Algorithm for Code Optimization on Grid Maps. **International Journal of Advanced Computer Science and Applications**, v. 13, n. 12, 2022.

LADEIRA, Ricardo de la Rocha. **Material didático**. Blumenau, SC, 2026.

LENGYEL, Eric. **Voxel-Based Terrain for Real-Time Virtual Simulations**. Doctoral dissertation—*[S.l.: S.n.]*.

LILJA, David J. **Measuring Computer Performance: A Practitioner's Guide**. Cambridge: Cambridge University Press, 2000.

LORENSEN, William E.; CLINE, Harvey E. Marching Cubes: A High Resolution 3D Surface Construction Algorithm. *In:* : SIGGRAPH '87.New York, NY, USA: Association for Computing Machinery, 1987.

PARDEDE, Sara Lutami *et al*. A review of pathfinding in game development. **Journal of Computer Engineering**, v. 1, n. 01, p. 47–56, 2022.

PATEL, Amit J. **Making maps with noise functions**. Disponível em: <https://www.redblobgames.com/maps/terrain-from-noise/#elevation-redistribution>. Acesso em: 7 maio. 2026.

PERLIN, Ken. An Image Synthesizer. *In:* : SIGGRAPH '85.New York, NY, USA: Association for Computing Machinery, 1985. Disponível em: <https://doi.org/10.1145/325334.325247>

POHL, Ira. Heuristic Search Viewed as Path Finding in a Graph. **Artificial Intelligence**, v. 1, n. 3–4, p. 193–204, 1970.

RUSSELL, Stuart; NORVIG, Peter. **Inteligência Artificial: uma abordagem moderna**. 3. ed. Rio de Janeiro: Elsevier, 2013.

SNOOK, Greg. Simplified 3D Movement and Pathfinding Using Navigation Meshes. *In:* DELOURA, Mark (Org.). **Game Programming Gems**. Rockland, MA: Charles River Media, 2000. p. 288–304.

STURTEVANT, Nathan R. Benchmarks for Grid-Based Pathfinding. **IEEE Transactions on Computational Intelligence and AI in Games**, v. 4, n. 2, p. 144–148, 2012.

VRIES, Joey de. **Learn OpenGL: Enjoy learning modern OpenGL**. Disponível em: <https://learnopengl.com/>. Acesso em: 18 abr. 2026.

WILLIAMS, Anthony. **C++ Concurrency in Action**. 2nd. ed. *[S.l.]:* Manning Publications, 2019.

ZIPPED. **C++: Perlin Noise Tutorial**. Disponível em: <https://www.youtube.com/watch?v=kCIaHqb60Cw>. Acesso em: 7 maio. 2026.